

ball with radius $\sqrt{M_Q^2 + M_K^2}$ and the distance of the transformed vectors decreases linearly with the inner product of the original vectors:

$$\|F(q_i) - G(k_i)\|_2^2 = 2 \cdot (M_Q^2 + M_K^2 - q_i \cdot k_i). \quad (5)$$

Note that the Euclidean distance of the transformed vectors depends only on the inner product of the original vectors and not on individual norms $\|q_i\|_2$ as in previous work [35, 34, 33]. We include details of comparison to the numerous prior asymmetric transformations in the Appendix.

Adaptive Clustering: The final step of SMYRF is to use the hashed values to create *balanced* clusters. These are created by forming balanced hash buckets where every group is assigned the same number of query and key vectors. We modify the E2LSH [35] hashes to create balanced clusters as follows: Instead of rounding the E2LSH to an integer value as in [35], we adaptively set the boundaries of the 1-d hashed space to ensure the same number of query and key vectors per interval. Computationally wise, this only requires sorting the hashes. We explain the mathematical details of our adaptive clustering scheme and the differences with E2LSH in the Appendix.

Computational Complexity and speedups: For notational simplicity we assume $|\mathcal{Q}| = |\mathcal{K}| = N$. The total time and memory complexity of SMYRF is $O\left(H \cdot N \cdot \log N + H \cdot \frac{N^2}{L}\right)$, where: H denotes hashing rounds, N number of query/key vectors and L number of clusters. For most of our experiments we choose $L = O(N)$, $H = O(1)$, and thus complexity is $O(N \log N)$. Even though we obtain optimal complexity for $L = O(N)$, $H = O(1)$, both L, H are parameters that can be tuned to satisfy the desired memory-performance trade-off. Regarding speed, SMYRF accelerates a lot attention as sequence length increases. For example, for sequence length 2048, SMYRF-BERT offers $\approx 20\%$ speedup, while for 4096 speedup increases to $\approx 50\%$. We include detailed speed plots for applying SMYRF to BERT in the Appendix.

4 Experiments

4.1 Pre-trained models

We first illustrate that SMYRF is an excellent drop-in replacement for pre-trained dense attention. We show significant memory benefits for relatively small performance drop, *with no training at all*. We use a pre-trained² BigGAN, which is a state-of-the-art model in Image Generation for ImageNet [37]. BigGAN has a single attention layer at resolution 64×64 (4096 queries). We replace BigGAN’s dense attention with a SMYRF layer at the same resolution, with no other modifications. Figure 1 illustrates images generated by SMYRF-BigGAN for different memory savings, ranging from 99.44% (first column) to 50% (one to last column). Last column shows generated images using the dense attention layer (100% memory). As shown, SMYRF enables a new tradeoff in the design space. We can drastically reduce attention memory by 93.75% with a small degradation or select any other point in this tradeoff depending on hardware specifications. We report a few Inception [38] and FID [39] scores for different memory savings in Table 1. We emphasize that no further modification was made to this model other than replacing the attention layer. By shrinking 50% the memory requirements of attention, SMYRF maintains 98.2% of Inception performance without any training. In the Appendix, we also include visualizations of clustering assignments in real-world images.

4.2 Finetuning pre-trained models

In this section, we *finetune* pre-trained models with SMYRF. We show that finetuned SMYRF models, with 50% memory reduction, can outperform dense attention. We also show that even with more aggressive memory-shrinking, up to 97%, SMYRF maintains a relatively good performance.

We train SMYRF-BERT (base) on GLUE [25, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51] benchmark, using sequence length 128. We compare the following five models: (i) BERT [6] (base), (ii) SMYRF-BERT (base) with 50% memory reduction (2nd row), (iii) SMYRF-BERT (base) with 25% memory reduction (3rd row), (iv) BERT (base) with input sequences truncated to 64 tokens (50% memory reduction, 4th row), (v) BERT (base) with input sequences truncated to 32 tokens

²Since BigGAN’s official checkpoints are not publicly available, we use the authors’ open-source, PyTorch [36] pre-trained models: <https://github.com/ajbrock/BigGAN-PyTorch>

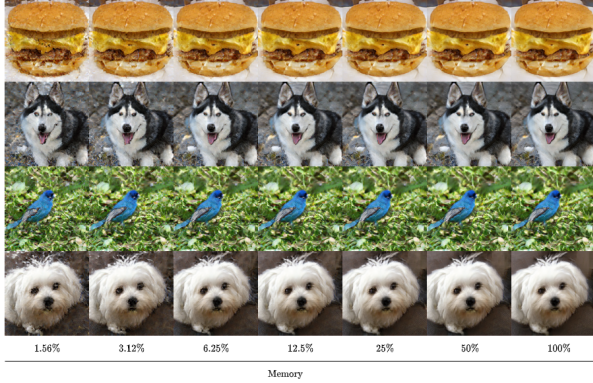


Figure 1: Images generated by SMYRF-BigGAN. The model is initialized with the weights of a pre-trained BigGAN on ImageNet (no further training). We show images for memory reduction ranging from 98.44% (first column) to 50% (one to last column). Last column shows generated images by BigGAN with dense attention.



Figure 2: Generated images from SMYRF-BigGAN on Celeba-HQ-128. Attention at 128×128 . The trained model uses 50% less memory compared to original BigGAN.

	Memory	Rounds	C	Inception	FID
BigGAN	100%	1	4096	93.79 ± 1.96	11.30
SMYRF-BigGAN	50%	32	64	91.91 ± 2.65	12.18
		64	32	92.09 ± 1.83	12.18
		128	16	91.59 ± 1.83	12.10
	25%	32	32	87.90 ± 1.90	13.34
		64	16	88.45 ± 1.70	13.45
		128	8	89.61 ± 1.63	13.19
	12.5%	32	16	81.67 ± 1.97	16.08
		64	8	82.87 ± 1.82	16.00
		128	4	82.10 ± 2.06	16.03

Table 1: Effect of SMYRF attention approximation on a pre-trained BigGAN (with no training). Rounds denote the number of LSH hashes and C the number of queries per cluster.

(25% memory reduction, 5th row). We summarize results on Table 2. Remarkably, SMYRF-BERT (slightly) **outperforms** original dense attention, while using 50% less memory. We also underline that SMYRF-BERT with 25% of original memory, maintains $\approx 99\%$ of original model performance, while the BERT-model that uses the same memory (last row) maintains only $\approx 89\%$.

To demonstrate that SMYRF scales for larger models, we also run experiments with **SMYRF-BERT large** to a subset of the GLUE tasks. Specifically, SMYRF-BERT large obtains **60.4%** performance (Matthew’s Correlation) in the CoLA task and **90.2%** (accuracy) in the QQP task. Both scores are significantly improved compared to the scores of the SMYRF-BERT base model, which shows that the approach scales to models with more attention layers. The corresponding scores of BERT large are 60.5% and 89.3% which are on par with the SMYRF performance for that model.

Since GLUE [25] datasets contain mostly short inputs, we also experiment on the IMDB [52] dataset, using sequence length 512 tokens³. We experiment with SMYRF-BERT (base) and we report results for various configurations (memory savings, hashing rounds and cluster size). To support our argument that our algorithm is a drop-in replacement to *any* dense attention layer, we also include some results for RoBERTa [9] (base). Results are summarized in Table 3. Notably, SMYRF-BERT maintains 97.2% of dense attention performance for 87.5% memory savings.

³Note that for fair comparison with dense attention, we train SMYRF layers and dense layers on the same sequence length, following the comparison scheme of Reformer [18]. As previous work has shown [28], training on IMDB (and other long-input datasets) with bigger sequence length can help performance.

In Table 4, we provide results for a Back-and-Forth procedure: we finetune with SMYRF and then for inference we use dense attention. By doing that, we observe performance almost equivalent to training with dense attention, while saving computational resources with SMYRF training. This indicates interchangeability between SMYRF and dense attention, which has not been previously reported. We use it to train in a memory efficient manner and obtain maximum final performance.

	Avg.	#	C	CoLA	MNLI-m/mm	MRPC	QNLI	QQP	RTE	SST-2	STS-B
BERT ₁₂₈	82.69	1	1	57.83	84.43/84.68	88.41	91.31	89.70	65.70	93.46	88.73
SMYRF-	83.12	2	32	58.79	85.02 /84.27	87.69	91.14	89.72	68.59	93.23	89.65
BERT	81.74	2	16	58.90	82.86/83.49	85.72	89.53	89.33	64.98	93.12	87.75
BERT ₆₄	81.57	1	64	58.80	82.34/82.47	87.02	90.48	89.69	61.73	93.00	88.64
BERT ₃₂	73.56	1	32	56.40	64.51/63.41	77.89	79.81	88.59	55.23	92.66	83.53

Table 2: Results on GLUE [25] (dev). # : hashing rounds. C : the number of queries per cluster. SMYRF outperforms BERT while using 50% less memory in each of the 12 attention layers.

	Dataset	Memory	Accuracy	Rounds	Cluster
BERT	IMDB	100%	94.12%	1	512
		50%	92.64%	8	32
		25%	92.52%	16	8
		12.5%	91.46	8	8
		6.25%	88.78%	8	4
		3.125%	87.49%	4	4
RoBERTa		100%	94.96%	1	512
SMYRF-RoBERTa		50%	93.72	8	32

Table 3: Finetuning BERT [6] (base) and RoBERTa [9] (base) on IMDB dataset for various configurations. For SMYRF models, we **train** and **evaluate** with SMYRF.

	Dataset	Memory	SMYRF Inference	Accuracy
RoBERTa	IMDB	100%	X	94.96%
SMYRF-		50%	X	93.72%
RoBERTa			✓	94.62%
BERT		100%	X	94.12%
SMYRF-		50%	X	92.64%
BERT			✓	93.54%

Table 4: Interchangeability of SMYRF and dense attention. We **train** with SMYRF and **evaluate** with *dense attention* for lightweight training and maximum performance.

4.3 Training from scratch

We also include experiments for networks trained from scratch. This shows that a non-pretrained model can learn with randomly initialized, SMYRF layers. Initially, the random weights produce less sparsity. However, the model quickly learns to create sparse attention maps and learning under our framework is possible. We use BigGAN [1] as the underlying model (see Appendix for details). We conduct our experiments on Celeba-HQ [29], which contains 30K images of celebrities at resolution 1024×1024 . We choose Celeba-HQ because: (i) images are in resolution higher than 128×128 , (ii) our budget is limited and Celeba-HQ requires much less training steps compared to ImageNet [37]. With SMYRF, we move attention from 64×64 resolution to 128×128 and train with 50% less memory than dense attention. In Table 5, we report FID for BigGAN and SMYRF-BigGAN after 120K steps training on Celeba-HQ-128 (downsampled to 128×128). SMYRF-BigGAN *outperforms* BigGAN’s FID by **3.95%**. Generated images from our model are shown in Figure 2. We finally move the attention layer to resolution 256×256 (65k length) and we successfully train on Celeba-HQ-256 for 120K steps on a single TPU v3-8. As far as we know, no other GAN has been trained with attention in higher resolution than this. Details and generated images are included in the Appendix.

	Resolution	Attention	Memory	Rounds	C	FID
BigGAN	128×128	64×64	100%	1	4096	26.06
SMYRF-BigGAN		128×128	50%	4	2048	25.03

Table 5: Results on BigGAN training on Celeba-HQ-128 for 120K steps. Moving attention from 64×64 to 128×128 helps performance: FID decreases from 26.06 to **25.03**. Memory percentages in this Table have as reference the memory a dense attention layer would use at the given resolution.

Model	IMDB (3 epochs)
SMYRF-RoBERTa	93.7%
E2LSH	89.3%
Reformer	88.7%

Table 6: LSH ablation experiment. The E2LSH model corresponds to the SMYRF-RoBERTa model using the E2LSH [35] hashing scheme instead of the asymmetrical transformations. The Reformer model corresponds to running SMYRF-RoBERTa with the cross polytope LSH [53] scheme, which is used in the Reformer [18] paper.

4.4 Comparison with other efficient attention techniques

To validate the effectiveness of the proposed asymmetrical transformations, we replace SMYRF’s hashing scheme with the E2LSH [35] scheme and the cross-polytope LSH [54] scheme of the Reformer and we evaluate all models on the IMDB [52] dataset, after training for three epochs. The results are summarized in Table 6. As shown, the asymmetrical transformations of SMYRF largely outperform all the other LSH schemes. This is expected since by design SMYRF tries to form clusters that maximize the inner products between queries and keys, while E2LSH and Reformer try to minimize euclidean distance and angular distance respectively, which is not the best objective when dealing with queries and keys with different vector representations and arbitrary norms.

To compare with the Longformer [28], we evaluate SMYRF on the Hyperpartisan News Detection [55] dataset. For this task, Longformer reports 94.8% accuracy with 4096 context-length. SMYRF obtains **97.2%** performance while only using 512 tokens. Longformer slightly outperforms (for $\approx 1\%$) SMYRF in the IMDB dataset but it uses 8 times more tokens to achieve that. Unfortunately, the available RoBERTa [9] models have been trained with maximum positional embeddings at 512 tokens and thus we cannot determine whether bigger sequence lengths would favor SMYRF. Nevertheless, SMYRF performs on par with other efficient attention techniques without requiring any pre-training.

5 Related work

The fact that attention maps of pre-trained layers are sparse is well-known [15, 16, 3, 17, 56, 57]. Relevant research to our work includes efforts to leverage that sparsity by limiting attention of each element to a subset of the original sequence. [23] proposes to limit attention to a sliding window around each element. Even though this simple idea is a strong baseline due to locality, this method is usually outperformed [20, 18, 19] by data-driven methods for assigning to each query the keys it will attend to. One recent research work that performs well with pre-defined sparsity is Longformer [28]. Longformer has been shown to perform well in downstream tasks after pre-training for 65K gradient steps, resuming MLM training of a pre-trained RoBERTa [9] model. However, this work requires custom GPU kernels that do not transfer across hardware (i.e. are not efficient on TPUs). SMYRF differs from Longformer in other important aspects as well: (i) SMYRF does not require (even though it might help) further pre-training before finetuning on downstream tasks. Therefore, SMYRF is a drop-in replacement of dense attention, while Longformer [28] requires some adaptation of the original dense attention. (ii) More importantly, the fixed sparsification idea used in Longformer [28] is fundamentally different from our idea of using clustering to approximate attention and (iii) SMYRF can be used interchangeably with dense attention while Longformer cannot. As we showed, a trained SMYRF attention lower can be converted back to a normal dense attention layer during inference.

There are three research works that are very relevant to ours since they also propose data-driven attention within each group: (i) the Reformer [18], (ii) the Sparse Sinkhorn Attention [20] paper and